A.Tejaswini-192471043

Q101. Graph Coloring Problem with Forbidden Colors per Vertex Develop a pseudocode using Backtracking to color a graph when each vertex has a list of forbidden colors that cannot be assigned to it. Clearly identify inputs (graph, color set, forbidden color list for each vertex) and output (valid coloring or failure). Design an algorithm that assigns colors vertex by vertex while checking adjacency and vertex-specific restrictions. Use loops, recursive calls, and color-checking conditions effectively during assignment. Apply pruning to reject branches where a color conflict or forbidden-color condition occurs. Ensure the pseudocode is clearly structured and easy to understand. Handle all possible cases and guarantee correctness of the final coloring.

Ans:

```
# Q101. Graph Coloring with Forbidden Colors per Vertex

# Backtracking solution

def is_safe(vertex, color, graph, coloring, forbidden):

 # Check forbidden color condition

 if color in forbidden.get(vertex, []):

 return False

 # Check adjacent vertex color conflict

 for neighbor in graph[vertex]:

 if coloring.get(neighbor) == color:

 return False

 return True

def graph_coloring_forbidden(graph, colors, forbidden):

 vertices = list(graph.keys())

 coloring = {}

 def backtrack(index):

 if index == len(vertices):

 return True

 vertex = vertices[index]
```

```
 for color in colors:
 if is_safe(vertex, color, graph, coloring, forbidden):
 coloring[vertex] = color
 if backtrack(index + 1):
 return True
 del coloring[vertex]
 return False
 if backtrack(0):
 return coloring
 else:
 return None
# Input
graph = {
 "A": ["B", "C"],
 "B": ["A", "C", "D"],
 "C": ["A", "B", "D"],
 "D": ["B", "C"]
}
colors = ["Red", "Green", "Blue"]
forbidden = {
 "A": ["Red"],
 "B": ["Green"],
 "C": [],
 "D": ["Blue"]
}
# Function call
result = graph_coloring_forbidden(graph, colors, forbidden)
```

```
# Output
if result is None:
 print("No valid coloring found")
else:
 print("Valid Coloring:")
 for vertex in result:
 print(vertex, "->", result[vertex])
```

Q102. Graph Coloring Problem with Precedence Order of Vertices

Write a pseudocode using Backtracking to color a graph by following a specified order of vertices instead of the natural numbering. Clearly define inputs (graph, number of colors, ordered vertex list) and output (valid coloring or no solution). Design an algorithm that colors vertices according to the given order while preserving adjacency constraints. Use loops, recursion, and conditional validation effectively to explore legal color assignments. Apply pruning whenever a color choice creates a conflict with already colored adjacent vertices. Present the pseudocode in a clear and logically consistent format. Ensure the solution handlesall ordered-coloring cases correctly and remains complete.

Ans:

```
# Q102. Graph Coloring with Precedence Order of Vertices
# Backtracking solution
def is_safe(vertex, color, graph, coloring):
 # Check adjacent vertex color conflict
 for neighbor in graph[vertex]:
 if coloring.get(neighbor) == color:
 return False
 return True
def graph_coloring_ordered(graph, num_colors, ordered_vertices):
 colors = list(range(1, num_colors + 1))
 coloring = {}
 def backtrack(index):
```

```
 if index == len(ordered_vertices):
 return True
 vertex = ordered_vertices[index]
 for color in colors:
 if is_safe(vertex, color, graph, coloring):
 coloring[vertex] = color
 if backtrack(index + 1):
 return True
 del coloring[vertex]
 return False
 if backtrack(0):
 return coloring
 else:
 return None
# Input
graph = {
 "A": ["B", "C"],
 "B": ["A", "C", "D"],
 "C": ["A", "B", "D"],
 "D": ["B", "C"]
}
num_colors = 3
ordered_vertices = ["C", "A", "D", "B"]
# Function call
result = graph_coloring_ordered(graph, num_colors, ordered_vertices)
# Output
if result is None:
```

```
 print("No solution exists")
else:
 print("Valid Coloring:")
 for vertex in ordered_vertices:
 print(vertex, "-> Color", result[vertex])
```